



PROJECT REPORT

Course: CSE215L(Programming Language-II Lab)

Section: 11

Faculty: Dr. Shamim Al Mamum Sir
Professor

Department of Electrical and Computer Engineering

PROJECT NAME: NSU Alumni HUB

Submitted by:

1. Name: Raiyan Choudhury

ID: 2531141042

2. Name: Tafsir Hasan

ID: 2523828642

3. Name: Hassan Shayer Palok

ID: 2524003042



Scan For Project

Project link :

https://github.com/Raiyanc01/NSU_ALUMNI_HUB/tree/main/javaswing

Submission Date: 23 August, 2026

Table of Contents

1. Introduction	4
1.1 Purpose / Objective	4
1.2 Problem Statement	4
1.3 Scope of the Project	4
2. Project Overview / Description	4
2.1 Key Features	4
2.2 Scope Boundaries.....	5
3. System Requirements.....	6
3.1 Hardware Requirements	6
3.2 Software Requirements.....	6
4. Design / Methodology	6
4.1 Startup Path	6
4.2 Persistence-to-Object Path.....	6
4.3 Object-Oriented Design.....	7
4.4 Flowchart	8
4.5 UML Diagram	9
5. Implementation.....	10
5.1 Authentication Workflow	10
5.2 Registration Workflow	10
5.3 Forgot Password Workflow	10
5.4 Swing Interface and Interaction Model.....	10
5.5 Event-Driven Behavior.....	11
5.6 Persistence and Data Management	11
5.7 Credential Protection	11
5.8 News Feed and Notice Board Flow	11
5.9 Real-Time Messaging Flow	12
5.10 Administration	12
5.11 Important Observations	12
6. Output / Testing	13
6.1 Validation and Exception Handling	13
6.2 Testing Evidence	13
6.3 Final Assessment.....	13
7. Challenges Faced & Solutions	14
7.1 Strengths	14
7.2 Observed Limitations and Improvement Directions.....	14
8. Conclusion.....	15
9. References	15
10. Appendix	15
10.1 Team Contributions.....	15

NSU Alumni HUB

1. Introduction

1.1 Purpose / Objective

NSU Alumni Hub is a local Java desktop application built to help North South University alumni stay connected: it combines profile management, a public news feed, an events calendar, a searchable alumni directory, live socket-based messaging, an official notice board, CV/resume submission, and an administrator console in a single Java Swing program.

The program starts through the Main class, which launches the GUI class. `GUI.guiexecute()` calls `accountManager.initStorage()` to prepare the local CSV account store and then opens the login window. After a successful login, GUI hands control to a new `MainDashboard` for the authenticated user, which loads that user's profile and renders the sidebar-driven workspace.

1.2 Problem Statement

Graduating students and alumni often lose touch with their department, their batch-mates, and university announcements once they leave campus, and there is rarely a single, lightweight place to see upcoming alumni events, reach a former classmate directly, or submit a CV for referrals. This project addresses that gap with a self-contained desktop tool that requires no external database, cloud account, or paid hosting — only a locally running chat server for the live-messaging feature.

1.3 Scope of the Project

- Register and authenticate alumni accounts with hashed passwords, a security question, and a three-attempt logout.
- Maintain a personal profile with department, batch, contact details and an uploadable profile photo.
- Provide a public news feed with posts, likes and threaded/nested comments.
- Provide an events listing that any member can post to and browse, with a details popup per event.
- Provide a searchable alumni directory with a profile-preview dialog for each member.
- Provide live one-to-one and public messaging through a dedicated socket chat server.
- Provide an official notice board seeded with default announcements.
- Provide local CV/resume upload and storage.
- Provide an administrator-only panel to manage events, registered users, feed posts and notices.

2. Project Overview / Description

The table below summarizes the project at a glance.

Attribute	Detail
Project type	Java desktop application
Technology	Java + Java Swing + TCP sockets
Persistence	Local CSV files (.csv)
Core areas	GUI event handling, file-based persistence, authentication, client/server networking, concurrency
Source basis	8 supplied Java source files

2.1 Key Features

Area	Implemented Capability	Main Components
Authentication	Username/password login with PBKDF2 hashing and a three-attempt, one-hour lockout.	accountManager
Registration	Creates local accounts after duplicate and required-field checks.	GUI, accountManager
Recovery	Security-question verification followed by a password reset.	GUI, accountManager
Profile	View and edit-by-upload of profile photo and personal/academic details.	MainDashboard, accountManager
News Feed	Create, edit, delete posts; like/unlike; nested reply comments.	FeedManager, MainDashboard
Events	Post, view, edit and delete alumni events with detail popups.	MainDashboard (in-memory list)
Alumni Directory	Table of registered alumni with a click-through profile preview.	MainDashboard, accountManager
Messaging	Public lounge and private one-to-one chat over a TCP socket server.	chatbox, ChatServer
Notice Board	View official notices; administrators can post, edit and delete.	NoticeManager
CV Upload	Select a PDF/DOCX file and copy it into a local cv/ folder.	MainDashboard
Admin	Tabs to manage events, users, feed posts and notices.	MainDashboard, accountManager, FeedManager, NoticeManager
Persistence	Credentials and application data stored as local CSV files.	accountManager, FeedManager, NoticeManager

2.2 Scope Boundaries

- No database or cloud backend appears in the supplied implementation; every store is a flat CSV file.
- No formal automated test suite is included in the supplied source set.
- The messaging feature requires ChatServer to be started separately from the main GUI application before chatbox can connect.
- Security claims are limited to what the code actually implements.

3. System Requirements

3.1 Hardware Requirements

- Any standard desktop or laptop computer capable of running the Java Virtual Machine (JVM).
- Minimum 4 GB RAM recommended for smooth Swing GUI performance.
- A free local TCP port (12345) for the chat server when messaging is used.

3.2 Software Requirements

- Java Development Kit (JDK) — required to compile and run the application.
- Java Swing (included in the standard JDK) — used for the entire graphical interface.
- A Java-capable IDE (e.g., Eclipse or IntelliJ IDEA) or command-line javac/java tools.
- No external database is required; persistence uses local .csv files. An internet connection is not required, but ChatServer must be running locally for the Messaging feature to work.

4. Design / Methodology

The project is not split into formal packages beyond a single javaswing package, but its responsibilities form a practical presentation, control, persistence, and networking structure.

Area	Responsibility	Examples
Presentation	Windows, dialogs, panels, tables and navigation.	GUI, MainDashboard, chatbox
Application control	Startup, current-user state and dashboard composition.	Main, GUI
Persistence	Reading, writing and reconstructing CSV records.	accountManager, FeedManager, NoticeManager
Networking	Socket server/client for real-time chat.	ChatServer, chatbox
Protection	Password hashing, salting and lockout tracking.	accountManager

4.1 Startup Path

Main.main() → GUI.guiexecute() → accountManager.initStorage() → show Login Window → authenticate() → new MainDashboard(username)

4.2 Persistence-to-Object Path

alumni_users.csv → accountManager.getUserProfile(username) → String[] record → MainDashboard profile / directory rendering

accountManager, FeedManager and NoticeManager each read their CSV file with `Files.readAllLines()`, split each line on commas, and hand back plain `String[]` rows. There is no dedicated domain-object model (no `Post`, `Notice` or `User` class) — the Swing panels work directly with these row arrays, which keeps the code simple but pushes all field-order knowledge into the presentation layer.

4.3 Object-Oriented Design

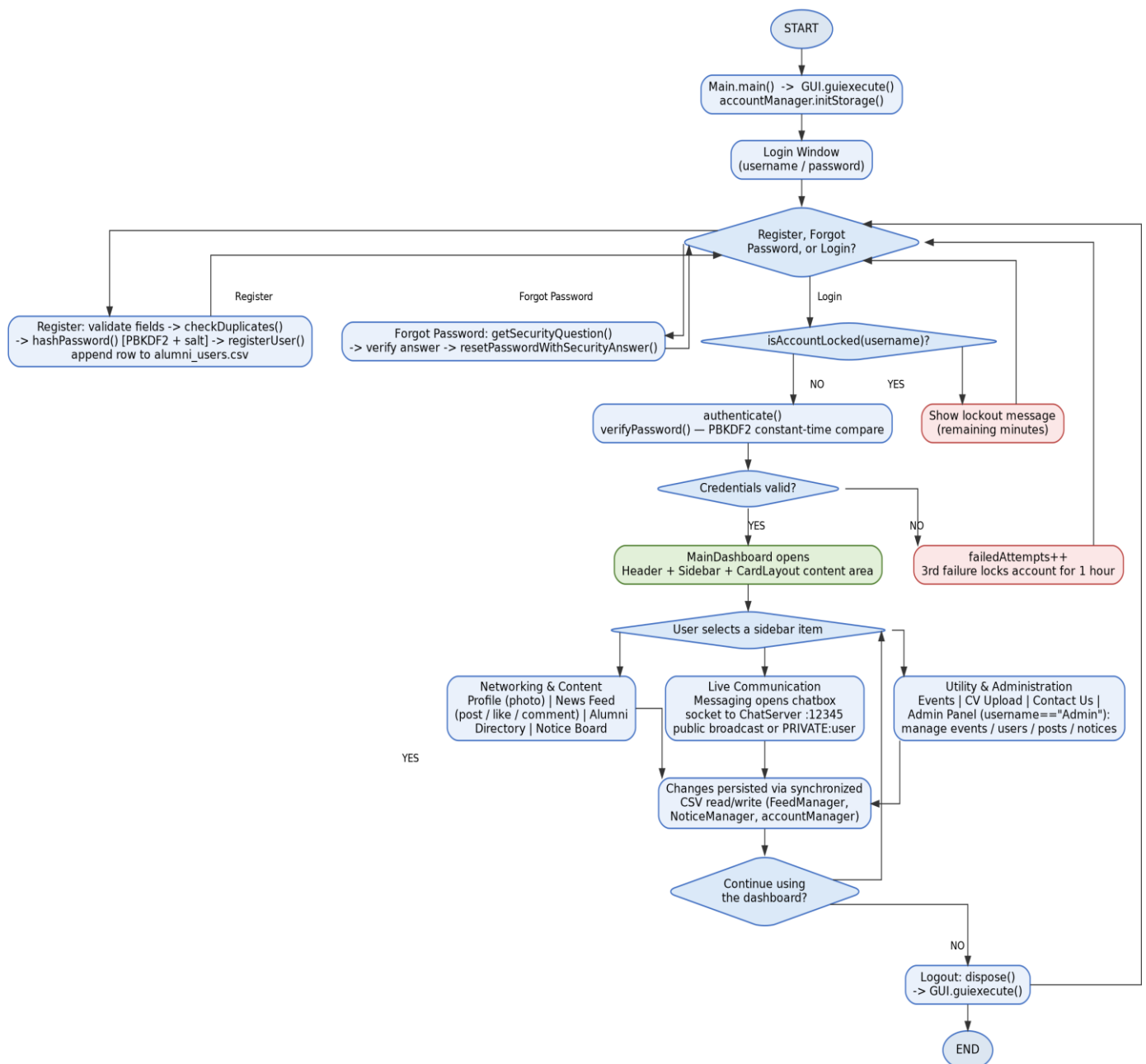
The project favors static utility classes and Swing subclassing over a custom class hierarchy. The concepts that are present are used in real, working code paths rather than being purely decorative.

Concept	Evidence	Application Use
Inheritance	MainDashboard extends JFrame; chatbox extends JFrame.	Reuses Swing's window, layout and event-dispatch machinery.
Interface implementation	ChatServer.ClientHandler implements Runnable.	Each connected client is serviced on its own thread.
Polymorphism / overriding	Overrides of run(), windowClosing(), mouseClicked() and actionPerformed().	Custom behavior is plugged into Swing's and Java's callback frameworks.
Encapsulation	Private static fields and a private nested AccountLockoutInfo class in accountManager.	Lockout state and CSV file names are hidden behind public static methods.
Anonymous / nested classes	ClientHandler (static nested), AccountLockoutInfo (private static nested), WindowAdapter/MouseAdapter subclasses.	Groups related state and behavior close to where it is used.
Concurrency primitives	ConcurrentHashMap, CopyOnWriteArrayList, synchronized methods, SecureRandom.	Thread-safe lockout tracking and thread-safe broadcast to many chat clients.

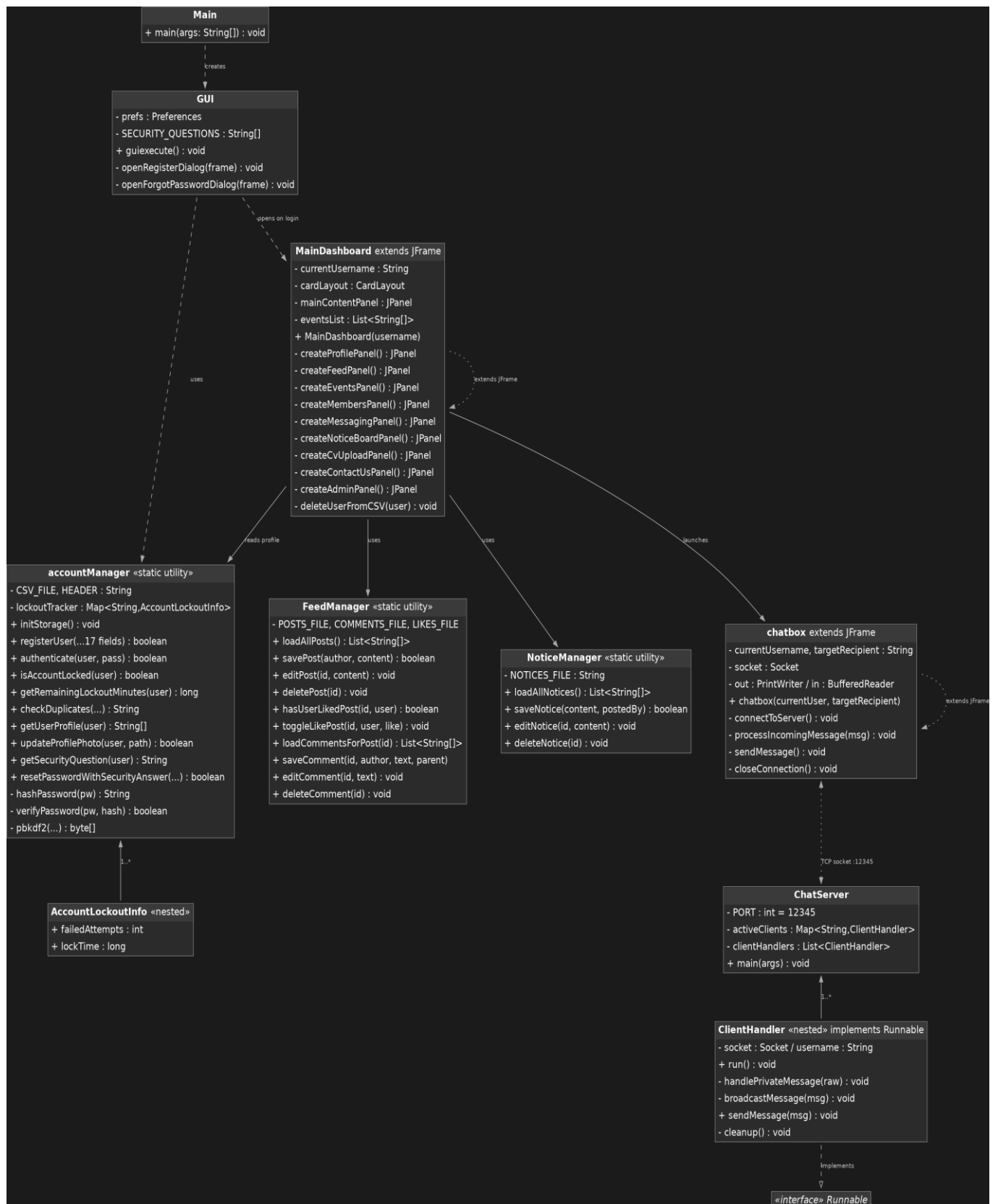
accountManager also owns the domain logic for password hashing (PBKDF2WithHmacSHA256), duplicate checking and lockout timing, keeping those behaviors close to the CSV data they operate on rather than scattering them across the GUI classes.

4.4 Flowchart

NSU Alumni Hub — System Flowchart
High-level flow based on the supplied Java implementation



4.5 UML Diagram



5. Implementation

5.1 Authentication Workflow

Input → isAccountLocked() check → authenticate() → loadUserPasswords() → verifyPassword() (PBKDF2 compare) → load profile → open MainDashboard

- Passwords are hashed with PBKDF2WithHmacSHA256 (65,536 iterations, 256-bit key) using a random 16-byte salt from SecureRandom; the salt and hash are stored together, Base64-encoded, separated by a colon.
- verifyPassword() recomputes the hash with the stored salt and compares it byte-by-byte with an XOR accumulator, avoiding an early-exit comparison.
- Three failed login attempts for a username lock that account for one hour; the remaining lockout time is shown to the user and tracked in a ConcurrentHashMap keyed by username.
- Successful authentication clears the lockout tracker for that user and opens MainDashboard with the authenticated username.

5.2 Registration Workflow

Form input → required-field check → checkDuplicates() (username / NSU ID / mobile / email) → hashPassword() → registerUser() appends a row to alumni_users.csv

- The registration dialog collects 17 fields, including a security question chosen from a fixed list and a profile-photo path from a JFileChooser.
- checkDuplicates() rejects the submission if the username, NSU ID, mobile number or email already exists in the CSV file.
- On success the CSV row is appended with StandardOpenOption.APPEND, and every field is passed through a clean() helper that strips embedded commas.

5.3 Forgot Password Workflow

Username entered → getSecurityQuestion() fetched → answer compared → resetPasswordWithSecurityAnswer() re-hashes and rewrites the row

- The security answer is stored and compared as lower-cased plain text, not hashed — it is a weaker check than the password itself.
- A successful reset also clears any existing lockout entry for that username.

5.4 Swing Interface and Interaction Model

The user interface is implemented entirely with Java Swing. MainDashboard is the largest presentation/controller class: it builds the header bar, the sidebar navigation, and every card of the CardLayout-driven content area, including the admin console.

Component	Role
JFrame	Login/registration window (GUI) and the main dashboard (MainDashboard), plus each chatbox window.
JDialog	Registration, forgot-password, event details, edit-event and profile-preview dialogs.
CardLayout	Switches the dashboard's main content panel between Profile, Feed, Events, Directory, Messaging, Notices, CV Upload, Contact Us and Admin.
JTable	Admin event/user/post/notice tables and the alumni directory table.

JPopupMenu	Per-post and per-comment Edit/Delete menus on the news feed.
JOptionPane	Confirmation, error and status dialogs across nearly every workflow.

5.5 Event-Driven Behavior

- Buttons use ActionListener callbacks, largely written as lambda expressions.
- MouseAdapter handles row clicks on the events table, the alumni-directory table, and link clicks in the Contact Us panel (opened with Desktop.browse()).
- A WindowAdapter in chatbox closes the socket connection when the chat window is closed.
- After a successful post, comment, like, event or notice change, the relevant panel is rebuilt and re-added to the CardLayout so the update is visible without a manual refresh.

5.6 Persistence and Data Management

accountManager, FeedManager and NoticeManager each provide local persistence using java.nio.file.Files, reading with Files.readAllLines() and writing with Files.write(), including StandardOpenOption.APPEND for new rows. Every public method that touches a file is declared synchronized to reduce corruption from concurrent Swing callbacks.

5.7 Credential Protection

Control	Implementation	Assessment
Password hashing	PBKDF2WithHmacSHA256, 65,536 iterations, per-user random salt	Reasonably strong for a coursework project
Attempt limit	Three failed attempts lock the account for one hour	Basic brute-force friction
Security answer	Stored and compared as lower-cased plain text	Weaker than the password path — a clear improvement target
Comparison	Manual XOR-accumulator byte comparison	Reduces (but does not fully guarantee) timing leakage
Admin gate	Admin tab shown only when the username equals "Admin"	Simple, hard-coded role check rather than a role field

5.8 News Feed and Notice Board Flow

Active posts → FeedManager.loadAllPosts() → posts.csv rows rendered newest-first → like/comment actions update likes.csv / comments.csv → panel reload

- Comments are stored with a parentId column, and renderCommentTree() recursively renders replies indented under their parent to build a simple threaded discussion.
- NoticeManager seeds three default notices the first time notices.csv does not exist, then reads/writes further notices the same way as posts.

5.9 Real-Time Messaging Flow

ChatServer.main() → ServerSocket(12345) accept loop → new ClientHandler thread per connection → REGISTER:username handshake → broadcast or PRIVATE:user:text routing

- ChatServer keeps two thread-safe collections: a ConcurrentHashMap of username → ClientHandler for direct lookup, and a CopyOnWriteArrayList of all handlers for broadcasting.
- A plain chat line is broadcast to every connected client; a PRIVATE:recipient:message line is routed only to that recipient's handler, or answered with a "currently offline" system message if the recipient is not connected.
- chatbox (the Swing client) opens its socket on a background thread, sends REGISTER:username, and continuously reads incoming lines with SwingUtilities.invokeLater() used to keep all UI updates on the Event Dispatch Thread.

5.10 Administration

- The Admin Panel is only added to the sidebar and CardLayout when the logged-in username equals "Admin" (case-insensitive).
- Manage Events lets the admin edit or delete any in-memory event; Manage Users lists every row of alumni_users.csv and can permanently delete a non-admin account (the CSV is rewritten to a temp file and swapped in).
- Moderate Feed Posts and Manage Notices list every row from FeedManager / NoticeManager and reuse the same edit/delete/post operations that the CSV managers expose to the ordinary panels.

5.11 Important Observations

- Events are stored only in an in-memory List<String[]> inside MainDashboard, so posted or edited events do not survive an application restart, unlike users, posts and notices, which are all persisted to CSV.
- toggleLikePost()'s unlike path deletes every row in likes.csv that matches the postId, which removes the like from every user who liked that post, not just the current user — a bug worth fixing before this feature is considered reliable.
- The simple comma-separated format has no formal CSV escaping beyond a manual clean()/sanitize() pass that strips commas and newlines from free-text input.
- Profile photos and CVs are copied into local profile_pics/ and cv/ folders and referenced by absolute file path, which is not portable across machines.

6. Output / Testing

6.1 Validation and Exception Handling

Standard Java exceptions and explicit null/empty checks are used to keep invalid values from silently becoming application state.

Condition	Location	Purpose
Empty username/password	GUI login and registration	Block obviously incomplete submissions
Duplicate username / NSU ID / mobile / email	accountManager.checkDuplicates()	Prevent account collisions
Locked account	accountManager.isAccountLocked()	Block login attempts during the lockout window
IOException	All CSV file operations	Report persistence failures to the console/user without crashing
NumberFormatException-prone parsing	Post like-count parsing in MainDashboard	Guarded by the sanitized CSV format written elsewhere in the code

6.2 Testing Evidence

Reliability features visible in the code include lazy file initialization with sensible defaults (initStorage(), the seeded notices), input validation before every write, GUI confirmation/error dialogs, and synchronized methods around shared CSV files. No formal automated test suite (e.g., JUnit) was supplied with the source set; testing evidence here is based on these code-level safeguards rather than dedicated test cases.

6.3 Final Assessment

Area	Assessment
GUI	A broad, genuinely navigable Swing interface: header, sidebar, CardLayout content, tables, dialogs and popup menus.
Networking	A working multi-client chat server with public broadcast and private routing over plain TCP sockets.
Persistence	Functional local CSV persistence for accounts, posts, comments, likes and notices.
Security	Solid password hashing, but a plain-text security answer and hard-coded admin check leave room for improvement.
Testing	No formal automated test suite was supplied; reliability rests on inline validation.

7. Challenges Faced & Solutions

7.1 Strengths

- A single Swing dashboard cleanly hosts eight distinct feature areas through one CardLayout and a consistent sidebar.
- The chat feature is a genuine client/server implementation with its own thread-per-client model, not a simulated stand-in.
- Password storage follows a real, modern hashing scheme rather than a reversible transformation.
- Feed and notice moderation are integrated into a single admin console reachable only by the designated admin account.

7.2 Observed Limitations and Improvement Directions

Observation	Effect	Recommended Improvement
Events live only in memory	All posted/edited events are lost when the app restarts.	Persist events to a CSV file the same way posts and notices already are.
Like removal deletes all likers	toggleLikePost(false) removes every like row for a post, not just the current user's.	Match on (postId, username) when deleting a like row.
Security answer stored in plain text	A weaker check than the hashed password path.	Hash the security answer with the same PBKDF2 routine used for passwords.
Hard-coded "Admin" username	Role membership is a string comparison, not a stored role field.	Add a role column to alumni_users.csv and check that instead.
ChatServer must be started separately	Messaging silently fails to connect if the server process was not launched first.	Auto-launch ChatServer from Main, or show a clear connection-status indicator in chatbox.
Simple comma parsing	Embedded commas in free text can still shift columns despite the clean()/sanitize() pass.	Use proper CSV escaping (quoted fields) or a structured format.
Absolute file paths for photos/CVs	Uploaded files are not portable if the CSV or app folder is moved to another machine.	Store paths relative to the application directory.

8. Conclusion

NSU Alumni Hub is a substantial Java Swing desktop application that combines account management, a social news feed, an events board, an alumni directory, live socket-based messaging, a notice board, CV submission and administrative moderation in one program. Its central design choice is a set of static, file-backed utility classes — `accountManager`, `FeedManager` and `NoticeManager` — feeding straightforward Swing panels through `CardLayout`, alongside a genuinely separate client/server pair for real-time chat.

The project demonstrates practical use of encapsulation, class inheritance from Swing components, interface implementation for threading, and concurrent collections for shared state, all wired into an event-driven GUI rather than left as isolated exercises. The CSV-based persistence layer keeps user, feed and notice data across runs, while `ChatServer` independently keeps alumni connected in real time.

The main opportunities for improvement are persisting events the same way other data is persisted, fixing the like-removal bug, hashing the security answer, replacing the hard-coded admin check with a stored role, and making the chat server easier to discover and connect to. These changes would make the system more robust and maintainable without changing its core purpose.

9. References

1. Introduction to Java Programming- 8th Edition- Dr. Y Daniel Liang

10. Appendix

10.1 Team Contributions

Member	Source-Marked Contribution
Raiyan Choudhury	MainDashboard, ChatServer, chatbox, GUI, program design, debugging
Tafsir Hasan	NoticeManager
Hassan Shayer Palok	accountManager , FeedManager, Main